

Polymorphism (in Java)

Mladen Ivkovic

**COMP2221 Programming Paradigms –
Object-oriented Programming**



Durham
University

Join by
Web

PollEv.com
/mladenivkovic356



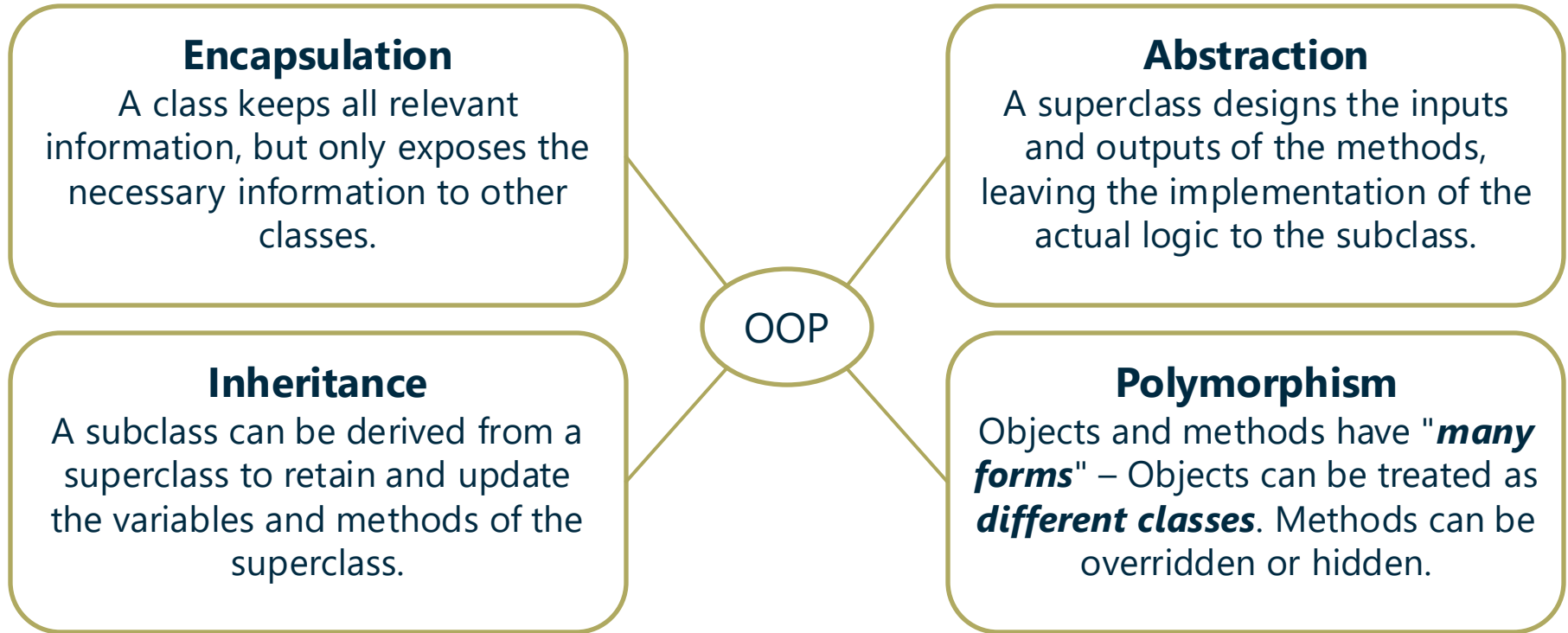
In This Lecture...

What Will You Learn in this Lecture?

- Polymorphism
- Revisiting
 - Method Overloading
 - Method Overriding
- Upcasting
- Downcasting

Polymorphism

The Four Principles of OOP



What is Polymorphism?

- Polymorphism means "**many forms**"
- Allows a single "thing" (object, method) to represent different "things" (classes/types, specialised methods)
- Achieved through:
 - **Method Overloading (Compile-time Polymorphism)**
 - **Method Overriding (Run-time Polymorphism)**
 - **Upcasting and Downcasting** (Type conversion in inheritance)
- **Benefits:** Improves code **reusability, flexibility, and maintainability**

Types of Polymorphism in Java

1. **Compile-time Polymorphism** (Method Overloading)
2. **Run-time Polymorphism** (Method Overriding)
3. **Upcasting and Downcasting** (Polymorphic Type Conversion)

Method Overloading (Compile-time Polymorphism)

- Multiple methods with the **same name but different parameters**
- Happens **within the same class**
- **Determined at compile-time**

```
class MathUtils {  
    int add(int a, int b) { return a + b; }  
    double add(double a, double b) { return a + b; }  
}
```

Method Overriding (Run-time Polymorphism)

- Allows a **subclass** to provide a specific implementation of a **superclass method**
- **Requires inheritance**
- Uses **@Override** annotation
- **Determined at run-time**

```
class Animal {  
    void makeSound() { System.out.println("Animal makes a sound"); }  
}  
class Dog extends Animal {  
    @Override  
    void makeSound() { System.out.println("Bark!"); }  
}
```

Polymorphism & Interfaces

- Interfaces enable multiple classes to **share a method signature**
 - Defines "contract"/"blueprint" **facilitating polymorphism**
 - Supports **loose coupling(**)** and **flexibility**

(**) *In software development, "**loose coupling**" means designing systems where components have **minimal interdependencies**, promoting flexibility and adaptability by reducing the impact of changes in one component on others.*

In this example: Changing contents of a class doesn't change the contents of the interface. The interface has no knowledge of the full contents of a class.

Polymorphism & Interfaces

- Interfaces enable multiple classes to **share a method signature**
 - Defines "contract"/"blueprint" **facilitating polymorphism**
 - Supports **loose coupling(**)** and **flexibility**

```
interface Animal {  
    void makeSound();  
}  
class Cat implements Animal {  
    public void makeSound() { System.out.println("Meow!"); }  
}
```

Polymorphism & Interfaces

- Example: We can use interfaces as a type!
- This lets us ***ignore what an object actually is as long as it behaves in a certain way***
 - "behaves in a certain way": Defined by the interface
- Code example on next slide.

Interfaces as a Type

- You can use Interfaces the same way you use types!
- (Full example on next slide)

```
interface Swimmable {  
    void printWaterDepth();  
}  
  
class Vehicle {  
    int nrPassengers;  
}  
  
class Boat extends Vehicle implements Swimmable {  
    float depth;  
    public void printWaterDepth() {  
        System.out.println("Depth below:"+depth);  
    }  
}
```

```
class Animal {  
    String name;  
}  
  
class Fish extends Animal implements Swimmable {  
    float depth;  
    public void printWaterDepth() {  
        System.out.println("Fish depth:"+depth);  
    }  
}
```

- You can use Interfaces the same way you use types!
- Note the use of swim() here: ***We don't care what it is, only that it behaves a certain way (Swimmable)***
- Runtime polymorphism: which printWaterDepth() method is called is decided at runtime

```
interface Swimmable {
    void printWaterDepth();
}

class Vehicle {
    int nrPassengers;
}

class Boat extends Vehicle implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Depth below:"+depth);
    }
}

class Animal {
    String name;
}

class Fish extends Animal implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Fish depth:"+depth);
    }
}
```

```
public class MyMain {

    // Using interface as a type
    public static void swim(Swimmable s) {
        s.printWaterDepth();
    }

    public static void main(String[] args) {

        Boat b = new Boat();
        swim(b);

        Fish f = new Fish();
        swim(f);

    }
}
```

Upcasting (Polymorphic Behaviour)

- **Upcasting:** Assigning (typecasting) a **subclass object** to a **superclass reference**
- All members of the superclass are accessible.
 - If methods are overridden: Overridden method will be selected at runtime.
- In this example: First implicit typecasting, then 2 lines below example of explicit typecasting

```
class Animal {
    void makeSound() { System.out.println("Animal sound."); }
}
class Dog extends Animal {
    void makeSound() { System.out.println("Bark!"); }
}

public class Main {
    public static void main(String[] args){
        Animal myPet = new Dog();           // Upcasting: Subclass object to superclass reference
        myPet.makeSound();                   // Calls Dog's method at runtime
        Animal myPet2 = (Animal) new Dog(); // explicit upcasting
    }
}
```


Upcasting (Polymorphic Behaviour)

- Why would you do this?
 - Flexibility, maintainability, reusability, clarity
 - **Example:** Method using superclass as argument accepts subclass objects as parameters
 - **Example:** Arrays, collections of subclass objects

Upcasting Usage: Example 1

```
class Animal {  
    void makeSound() { System.out.println("Animal sound."); }  
}  
  
class Ape extends Animal {  
    @Override void makeSound() { System.out.println("Ook"); }  
}  
  
class Bear extends Animal {  
    @Override void makeSound() { System.out.println("Roar"); }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() { System.out.println("Moo"); }  
}
```

```
public class Upcasting {  
  
    // class taking superclass object as argument  
    public static void makeSound(Animal a) {  
        a.makeSound();  
    }  
  
    public static void main(String[] args) {  
  
        Ape a = new Ape();  
        Bear b = new Bear();  
        Cow c = new Cow();  
  
        // passing subclass objects as argument  
        makeSound(a);  
        makeSound(b);  
        makeSound(c);  
    }  
}
```

Output:

Ook
Roar
Moo

- Compare to our previous example with interface!

Upcasting Usage: Example 2

```
class Animal {  
    void makeSound() { System.out.println("Animal sound."); }  
}  
  
class Ape extends Animal {  
    @Override void makeSound() { System.out.println("Ook"); }  
}  
  
class Bear extends Animal {  
    @Override void makeSound() { System.out.println("Roar"); }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() { System.out.println("Moo"); }  
}
```

```
public class Upcasting {  
  
    public static void main(String[] args) {  
  
        Ape a = new Ape();  
        Bear b = new Bear();  
        Cow c = new Cow();  
  
        // store objects of different type in  
        // single array of type Animal  
        Animal[] zoo = {a, b, c};  
  
        for (int i = 0; i < zoo.length; i++){  
            zoo[i].makeSound();  
        }  
    }  
}
```

Output:

Ook
Roar
Moo

- Compare to our previous example with interface!

Upcasting - Warning

- **Warning:** When upcasting, **only members of superclass are accessible**
 - Another reason why interfaces are so useful – it defines what ***must*** be available!

```
class Animal {
    void makeSound() {
        System.out.println("Animal sound.");
    }
}

class Ape extends Animal {
    @Override void makeSound() {
        System.out.println("Ook");
    }
    void climbTree() {
        System.out.println("Sitting in the tree.");
    }
}
```

```
public class Upcasting {

    public static void main(String[] args) {

        Ape a1 = new Ape();
        Ape a2 = new Ape();

        Animal[] zoo = {a1, a2};

        for (int i = 0; i < zoo.length; i++){
            zoo[i].makeSound(); // works
            zoo[i].climbTree(); // does NOT work!
        }
    }
}
```

Output:

Unresolved compilation problem: The method climbTree() is undefined for the type Animal

Downcasting (Reverting Upcasting)

- **Downcasting:** Converting (typecasting) a **superclass reference** back to a **subclass reference**
- Must be done **explicitly** using casting
- Gets us access to subclass members

```
Animal myPet = new Dog(); // upcasting  
Dog myDog = (Dog) myPet(); // downcasting  
myDog.makeSound(); // Works correctly
```

Downcasting Example

- Retrieve access to subclass members

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sound.");  
    }  
}  
  
class Ape extends Animal {  
    @Override void makeSound() {  
        System.out.println("Ook");  
    }  
    void climbTree() {  
        System.out.println("Sitting in the tree.");  
    }  
}
```

```
public class Downcasting {  
  
    public static void main(String[] args) {  
  
        Ape a1 = new Ape();  
        Ape a2 = new Ape();  
  
        Animal[] zoo = {a1, a2};  
  
        for (int i = 0; i < zoo.length; i++){  
            zoo[i].makeSound();           // works  
            // zoo[i].climbTree();        // does NOT work!  
  
            // instead, use downcasting:  
            ((Ape) zoo[i]).climbTree(); // works!  
  
            Ape a = (Ape) zoo[i];  
            a.climbTree();                // works!  
        }  
    }  
}
```

Downcasting – Warnings and Pitfalls

- Careful with conversions:
 - Subclass → superclass → subclass: **works**

```
Animal myPet = new Dog();           // upcasting
Dog myDog = (Dog) myPet();           // downcasting
myDog.makeSound();                   // Works correctly
```

```
Dog myDog = new Dog();               // instantiation

Animal myPet1 = (Animal) myDog;      // explicit upcasting
Animal myPet2 = myDog;               // implicit upcasting

Dog myDog1 = (Dog) myPet1();          // downcasting
myDog1.makeSound();                  // Works correctly

Dog myDog2 = (Dog) myPet2();          // downcasting
myDog2.makeSound();                  // Works correctly
```

Downcasting – Warnings and Pitfalls

- Careful with conversions:
 - superclass → subclass: **doesn't work** (*ideas why?*)

```
class Animal { }  
  
class Cow extends Animal {  
    float milkPerDay = -1.f;  
  
    Cow(float milkPerDay){  
        this.milkPerDay = milkPerDay;  
    }  
  
    void milk() {  
        System.out.println("Milk per day: " + milkPerDay);  
    }  
}
```

```
public class Downcasting {  
  
    public static void main(String[] args) {  
  
        Animal a = new Animal();  
        Cow c = (Cow) a; // doesn't work!  
        c.milk();  
    }  
}
```

Output:
**class Animal cannot be
cast to class Cow**

Downcasting – Warnings and Pitfalls

- Careful with conversions:
 - Subclass → superclass → different subclass: **doesn't work**

```
class Animal {  
    // code...  
}  
  
class Cow extends Animal {  
    // code...  
}  
  
class Dog extends Animal {  
    // code...  
}
```

```
public class Downcasting{  
  
    public static void main(String[] args) {  
  
        Cow c = new Cow();  
        Animal a = c;        // upcasting; works  
        Dog d = (Dog) a;     // doesn't work!  
    }  
}
```

Output:

class Cow cannot be cast to class Dog

The ``instanceof`` operator

- To avoid runtime errors when downcasting, we can make use of the ``instanceof`` operator
- Binary operator
- Returns ``true`` or ``false``
- Tests if an object is of a given type
 - You can use this to make sure downcasting is sensible!

The `instanceof` operator: Examples

- What is the output of this code?

```
class Animal {  
    // code...  
}  
  
class Cow extends Animal {  
    // code...  
}  
  
class Dog extends Animal {  
    // code...  
}
```

```
public class Downcasting{  
    public static void main(String[] args) {  
  
        Cow c = new Cow();  
        Animal a = c;  
        Animal a2 = new Animal();  
  
        System.out.println(c instanceof Cow);  
        System.out.println(c instanceof Animal);  
        System.out.println(c instanceof Dog);  
        System.out.println(a instanceof Cow);  
        System.out.println(a instanceof Animal);  
        System.out.println(a instanceof Dog);  
        System.out.println(a2 instanceof Cow);  
        System.out.println(a2 instanceof Animal);  
        System.out.println(a2 instanceof Dog);  
    }  
}
```

The `instanceof` operator: Examples

- What is the output of this code?

```
class Animal {  
    // code...  
}  
  
class Cow extends Animal {  
    // code...  
}  
  
class Dog extends Animal {  
    // code...  
}
```

```
public class Downcasting{  
    public static void main(String[] args) {  
  
        Cow c = new Cow();  
        Animal a = new Animal();  
        Animal a2 = c;  
  
        System.out.println(c instanceof Cow);  
        System.out.println(c instanceof Animal);  
        System.out.println(a instanceof Cow);  
        System.out.println(a instanceof Animal);  
        System.out.println(a instanceof Dog);  
        System.out.println(a2 instanceof Cow);  
        System.out.println(a2 instanceof Animal);  
        System.out.println(a2 instanceof Dog);  
    }  
}
```

Output:

```
true  
true  
false  
true  
false  
true  
true  
false
```

`instanceof` and Downcasting

- Use `instanceof` for safe downcasting by checking for the object's type

```
class Animal {  
    // code...  
}  
  
class Cow extends Animal {  
    void moo() {  
        System.out.println("Moo");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Bark");  
    }  
}
```

```
public class Downcasting{  
    public static void makeSound(Animal a){  
  
        // select behaviour based on subclass  
        if (a instanceof Cow) {  
            ((Cow) a).moo();  
        } else if (a instanceof Dog) {  
            ((Dog) a).bark();  
        }  
    }  
  
    public static void main(String[] args) {  
        Cow c = new Cow();  
        makeSound(c);  
  
        Dog d = new Dog();  
        makeSound(d);  
    }  
}
```

The `instanceof` operator: Warning

```
class Animal {  
    // code...  
}  
  
class Cow extends Animal {  
    // code...  
}  
  
class Dog extends Animal {  
    // code...  
}
```

```
public class Downcasting{  
    public static void main(String[] args) {  
  
        Cow c = new Cow();  
        System.out.println(c instanceof Dog);  
    }  
}
```

Output:
Incompatible conditional operand types Cow and Dog

- If the compiler can tell that `instanceof` will return `false`, it will throw an error

Conclusion

- **Polymorphism increases flexibility** in object-oriented programming
- **Method overloading** is determined **at compile-time**
- **Method overriding** is determined **at run-time**
- **Upcasting allows treating subclasses as superclass types**
- **Downcasting allows reverting to subclass behaviour (use carefully)**
- **Interfaces provide an alternative way to implement polymorphism**

Best Practices & Common Mistakes

- ✓ **Always use @Override annotation** when overriding methods
- ✓ **Ensure proper method signature** for overloading
- ✓ **Use polymorphism to simplify code, not overcomplicate it**
- ✓ **Avoid unnecessary overloading with similar parameter types**
- ✓ **Only downcast when necessary, and check type before casting**

The End

Any Questions?



Exercises

- Either tell me what the output of the code is
- Or tell me why the code is wrong and how to fix it

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound();  
  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound();  
  
    }  
}
```

Output:
Moo .

What is the output of this code?

```
class Animal {
    void makeSound() {
        System.out.println("Animal sounds.");
    }
}

class Cow extends Animal {
    @Override void makeSound(int repeat) {
        for (int i = 0; i < repeat; i++){
            System.out.println("Moo.");
        }
    }
}
```

```
public class Polymorphism {
    public static void main(String[] args) {

        Cow c1 = new Cow();
        Animal a1 = (Animal) c1;
        a1.makeSound();

    }
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound(int repeat) {  
        for (int i = 0; i < repeat; i++){  
            System.out.println("Moo.");  
        }  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound();  
  
    }  
}
```

Output:

The method makeSound(int) of Cow must override or implement a superclass method.

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    void makeSound(int repeat) {  
        for (int i = 0; i < repeat; i++){  
            System.out.println("Moo.");  
        }  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound(3);  
  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    void makeSound(int repeat) {  
        for (int i = 0; i < repeat; i++){  
            System.out.println("Moo.");  
        }  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound(3);  
  
    }  
}
```

Output:

The method makeSound() in the type Animal is not applicable for the arguments (int)

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    void makeSounds() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound();  
  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    void makeSounds() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal a1 = (Animal) c1;  
        a1.makeSound();  
  
    }  
}
```

This is why you should use @Override!

Output:
Animal sounds.

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal c2 = new Cow();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Animal a = pasture[i];  
            a.makeSound();  
        }  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Animal c2 = new Cow();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Animal a = pasture[i];  
            a.makeSound();  
        }  
    }  
}
```

Output:
Moo.
Moo.

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Animal c1 = new Cow();  
        Animal c2 = new Animal();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Cow c = (Cow) pasture[i];  
            c.makeSound();  
        }  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Animal c1 = new Cow();  
        Animal c2 = new Animal();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Cow c = (Cow) pasture[i];  
            c.makeSound();  
        }  
    }  
}
```

Output:

Moo.

class Animal cannot be cast to class Cow .

What is the output of this code?

Bonus question: Why does it print out `Moo.` first?

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Animal c1 = new Cow();  
        Animal c2 = new Animal();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Cow c = (Cow) pasture[i];  
            c.makeSound();  
        }  
    }  
}
```

Output:

Moo.

class Animal cannot be cast to class Cow .

What is the output of this code?

Bonus question: Why does it print out `Moo.` first?

Up/downcasting is **runtime polymorphism**. It only gets detected during runtime.

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Animal c1 = new Cow();  
        Animal c2 = new Animal();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            Cow c = (Cow) pasture[i];  
            c.makeSound();  
        }  
    }  
}
```

Output:

Moo.

class Animal cannot be cast to class Cow .

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
    void moo() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Cow c2 = new Cow();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            pasture[i].moo();  
        }  
    }  
}
```

What is the output of this code?

```
class Animal {  
    void makeSound() {  
        System.out.println("Animal sounds.");  
    }  
}  
  
class Cow extends Animal {  
    @Override void makeSound() {  
        System.out.println("Moo.");  
    }  
    void moo() {  
        System.out.println("Moo.");  
    }  
}
```

```
public class Polymorphism {  
    public static void main(String[] args) {  
  
        Cow c1 = new Cow();  
        Cow c2 = new Cow();  
  
        Animal[] pasture = {c1, c2};  
  
        for (int i = 0; i < pasture.length; i++) {  
            pasture[i].moo();  
        }  
    }  
}
```

Output:

The method moo() is undefined for the type Animal